第7章 第9节 传输与生命周期

- 来源: https://class.imooc.com/lesson/5281#mid=104938
- 阶段: 阶段三: 【工具与协议】让AI连接世界
- 周: 第7周 MCP：模型上下文协议
- 模块: MCP通信协议与实战
- 课节: 图文 1-2 传输与生命周期

1-2 传输与生命周期

第7章 第9节 传输与生命周期

> [!TIP] 阅读指南 上一节深入了解了 JSON-RPC 协议的消息格式和错误码。本节探讨这些消息如何在实际中传输，以及 MCP Server 从启动到关闭的完整生命周期。

9.1 传输层选择

JSON-RPC 定义了消息格式，但没有规定怎么传输。MCP 支持三种传输方式：

stdio（标准输入输出）

原理： 通过进程的标准输入输出（stdin/stdout）传输 JSON 消息。



> 什么是 stdio？ stdio 是「标准输入输出」（Standard Input/Output）的缩写。在计算机世界里，每个运行的程序默认都有三个通道：接收指令的 stdin（标准输入）、输出结果的 stdout（标准输出）以及报告错误的 stderr（标准错误）。MCP 利用这些天然存在的通道进行数据交换，就像两个人在用纸条（JSON 消息）通过门缝（stdin/stdout）互相传递信息。

典型场景：

```
# Client 启动 Server 进程
$ node mcp-server.js

# Server 通过 stdin 接收消息，通过 stdout 返回结果
```

stdio 的实现简单，任何语言都支持。它的进程隔离机制天然安全，适合本地工具。缺点是只能一问一答的单向传输，不适合 Web 场景。

这是MCP 最常用的方式 — Claude Desktop 就是用 stdio 启动 MCP Server。

SSE 传输层（基于 HTTP 的混合模式）

很多同学会混淆：MCP 是支持 HTTP 还是支持 SSE？ 答案是：它使用了一种「混合模式」，官方称为 SSE Transport。

原理： 由于传统的 HTTP 是严格的「一问一答」，服务端无法主动说话。因此，基于网络传输的 MCP 并没有单独使用 HTTP，而是将两种技术缝合在了一起：

- 指令上行（Client → → Server）： 使用标准的 HTTP POST 请求。
- 通知下行（Server → → Client）： 使用 SSE（Server-Sent Events）建立的长连接通道。

为什么不直接用传统 HTTP?

传统的 HTTP 是「不问不答」模式。但 MCP 协议要求服务端能主动通知客户端（比如：资源更新了、任务处理完了）。如果只用传统 HTTP，服务端无法主动「敲客户端的门」。

它不是两种模式，而是一种基于 HTTP 协议栈实现的双向方案。

```
┌──────────────────┐          ┌──────────────────┐
│    Web 应用                    MCP Server
│    (Client)                  (HTTP 服务)
│
│  1. POST 发指令 ── JSON ─▶  处理请求
│  2. 响应 (OK)   ◀─ JSON ──  处理结果
│
│  3. SSE 长连接  ◀── 推送 ──  主动通知
└──────────────────┘          └──────────────────┘
```

请求示例：

```
POST /rpc HTTP/1.1
Host: localhost:3000
Content-Type: application/json

{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "resources/list"
}
```

SSE 推送示例：

```
event: notification
data: {"jsonrpc":"2.0","method":"resources/updated","params":{...}}

1
2
```

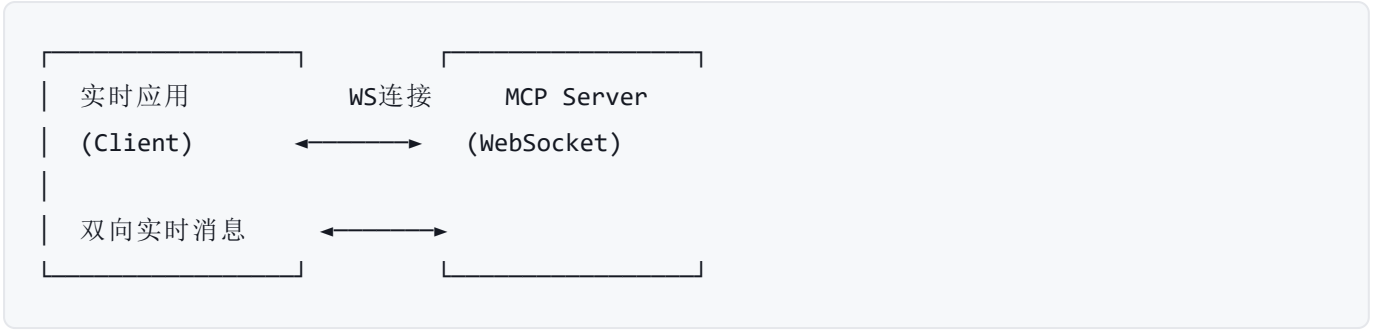
SSE 的传输方式适合 Web 集成，Server 可以主动推送通知，而且用 HTTP 工具做调试很方便。缺点是必须运行 HTTP 服务，比 stdio 模式复杂一些。

适用场景： 浏览器插件、Web 应用集成 MCP。

适用场景：

WebSocket

原理： 全双工通信，Client 和 Server 可以随时互发消息。



WebSocket 能实现真正的双向通信，延迟低，适合实时场景。缺点是实现较为复杂，需要保持长连接。

适用场景： 需要实时交互的协作应用、流式响应。WebSocket并不罕见，你使用的大部分网页版聊天程序都使用的WebSocket技术。

三种方式对比

传输方式	实现难度	双向通信	适用场景
stdio	★ 简单	全双工	本地工具、命令行应用
SSE Transport	★★ 中等	半双工(混合)	Web 应用、浏览器插件
WebSocket	★★★ 复杂	全双工	实时协作、流式交互

选择哪种传输方式要看场景：本地工具用 stdio，Web 服务用 HTTP+SSE，实时推送则用 WebSocket。

9.2 MCP Server 生命周期

一个 MCP Server 从启动到关闭，主要经历三个核心阶段。我们没必要纠结于每一个繁琐的字段，只需把握住最关键的「握手」与「交互」逻辑。

当 Client 启动 Server 后，第一件事就是交换彼此的「名片」和「技能清单」。

1. Client 发送请求：

```
{
  "method": "initialize",
  "params": {
    "protocolVersion": "2024-11-05", // 协议版本
    "capabilities": { "roots": {...}, "sampling": {...} } // Client 能做什么
  }
}
```

2. Server 返回响应：

```
{
  "result": {
    "protocolVersion": "2024-11-05",
    "capabilities": { "resources": {}, "tools": {}, "prompts": {} }, // Server 能提供什么
    "serverInfo": { "name": "my-server", "version": "1.0.0" }
  }
}
```

> 核心逻辑： 这一步决定了双方「能不能聊到一块去」。如果协议版本不匹配，或者 Server 声明不支持 Tools，那么后续的工具调用就无从谈起。

握手成功后，双方进入高效协作模式。主要有两种对话方式：

- 请求-响应（双向）：Client 主动出击。例如「请帮我读取这个 Resource」或「请执行这个 Tool」。Client 发出 id，Server 必须带回相同的 id。
- 通知（单向）：任何一方都可以发送。例如 Server 告诉 Client「资源列表更新了」，或者 Client 告诉 Server「用户取消了操作」。通知没有 id，不需要对方回复。

当 AI 应用退出时，会向 Server 发送取消通知。Server 接收到后，会像一位得体的管家一样：停止接受新任务、释放数据库连接、关闭文件句柄，最后安全退出进程。

完整生命周期示意图：

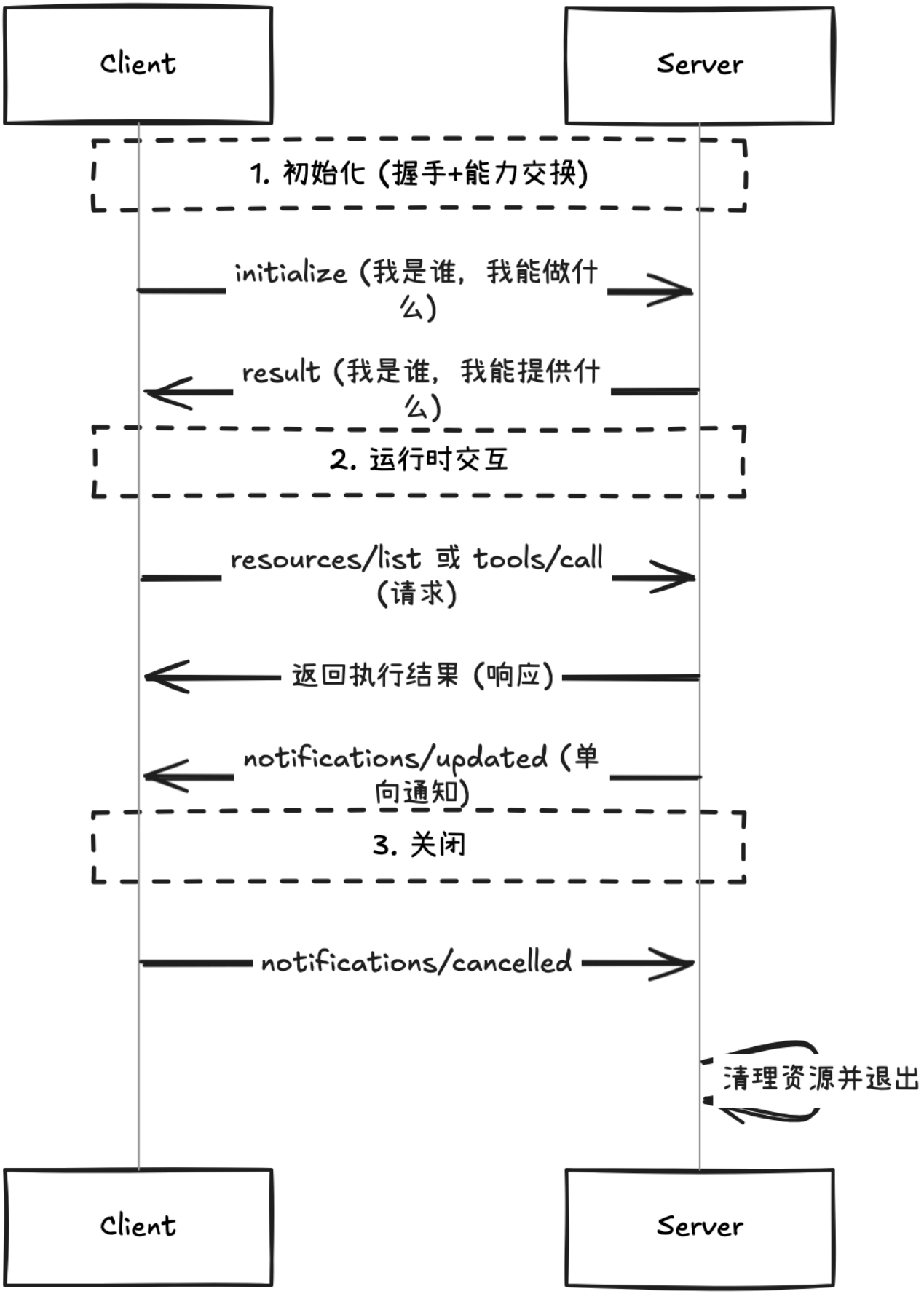
9.3 下一节预告

到这里，MCP 的底层通信协议已经梳理了一遍，从 JSON-RPC 的消息机制到各种传输层的选择。接下来进入实战环节——从零开发一个「游戏装备计算助手」，从定义工具到处理调用的全流程都会试一遍。

9.4 ■ 学点英语

中文	English	音标	说明
标准输入输出	stdio (Standard I/O)	/ˈstændərd aɪ oʊ/	通过进程stdin/stdout传输JSON-RPC消息的本地传输方式
服务器推送事件	SSE (Server-Sent Events)	/ˈsɜːrvər sent ɪˈvents/	HTTP长连接下Server主动推送数据的远程传输技术
初始化握手	Initialization Handshake	/ɪˌnɪʃəlaɪˈzeɪʃn ˈhændʃeɪk/	Client和Server交换协议版本和能力声明的连接建立阶段
优雅关闭	Graceful Shutdown	/ˈgreɪsfl ˈʃʌtdaʊn/	Server收到关闭通知后释放资源安全退出的过程

图片


Client
Server
1. 初始化 (握手+能力交换)
initialize (我是谁，我能做什么)
result (我是谁，我能提供什么)
2. 运行时交互
resources/list 或 tools/call (请求)
返回执行结果 (响应)
notifications/updated (单向通知)
3. 关闭
notifications/cancelled
清理资源并退出
Client
Server